OWASP A01:2021 – Broken Access Control

التصعيد الرأسي (Vertical Privilege Escalation Master Cheatsheet)

دليل هندسي كامل لاكتشاف وتحليل وتأمين ثغرات تصعيد الصلاحيات الرأسية وتجاوزات قيود الوصول على مستويات الادوار (RBAC).

1. المفهوم الأساسي والسبب الجذري (Core Concept & Root Cause)

التصعيد الرأسي (Vertical Escalation): يحدث عندما يتمكن مستخدم صاحب صلاحيات منخفضة (Standard User / Guest) من الوصول إلى وظائف أو مسارات أو إجراءات مخصصة للمدراء (Administrators).

السبب الجذري: الاعتماد على إخفاء خيارات التحكم في واجهة المستخدم (Client-Side Concealment) دون تطبيق فحص صارم للصلاحيات في الخادم (Server-Side Authorization Enforcement).

2. خطوات الاستطلاع والتحليل المنهجي (Methodology & Attack Steps)

- **الخطوة 1 (المصادقة):** تسجيل الدخول بحساب عادي (`Standard User`). *(تنبيه: في بعض الحالات قد تتوفر الوظائف دون مصادقة أصلاً (Unauthenticated Access).*
- **الخطوة 2 (الملفات المكشوفة):** فحص ملف `robots.txt/` والمستندات التقنية للمسارات المحجوبة عن محركات البحث.
- **الخطوة 3 (تحليل المصدر):** قراءة السورس كود (`Ctrl + U`) والبحث عن الملاحظات (`Comments`) واستخراج ملفات الجافاسكريبت (`.js`).
- **الخطوة 4 (الفحص وتخمين المسارات):** تنفيذ عملية تخمين شاملة للمسارات والوظائف الإدارية (`Fuzzing`) عند عدم وجود روابط صريحة.

3. أدوات الاستطلاع والفحص المتقدمة (Reconnaissance Tools)

LinkFinder:
تحليل ملفات JavaScript تلقائياً واستخراج المسارات المنسية والـ Endpoints المخفية داخل الكود.

Gobuster:
تخمين المجلدات والملفات باستخدام Wordlists بسرعة عالية للكشف عن استجابات مثل `200 OK` أو `403 Forbidden`.

JS-PathHunter:
استخراج التعليقات البرمجية المتروكة والمسارات بأسلوب Regex سريع ودقيق.

ffuf:
أداة Fuzzing عالية السرعة لاستبدال كلمة FUZZ في الروابط، الترويسات (Headers)، أو جسم الطلب (POST Data).

4. مصفوفة مستويات الصلاحيات والـ Roles (Role Mapping Matrix)

المستوى (Role Level)	الصلاحية التقديرية	سلوك النظام المستهدف	آلية الفحص والتحقق
Unauthenticated	عام (Public)	وصول للمستندات والصفحات العامة فقط.	اختبار الوصول المباشر للمسارات الإدارية بدون Session Cookie.
Standard User	مستشار/عميل عادي	إدارة البيانات الشخصية واستخدام الخدمات الأساسية.	محاولة طلب وظائف الإدارة مع إرسال Session المستخدم العادي.
Administrator	تحكم كامل (Full Control)	إدارة المستخدمين، التعديل، والحذف.	مطابقة الـ Endpoints المستجابة بين جلسة الأدمن وجلسة العميل.

5. التجاوزات المتقدمة وسيناريوهات API & JWT (Advanced Bypass Vectors)

أ. التلاعب بالترويسات (Header Injection):
استخدام ترويسات مثل `X-Original-URL: /admin` أو `X-Rewrite-URL: /admin` أو محاكاة الـ IP المحلي عبر `X-Forwarded-For: 127.0.0.1` لتجاوز فلاتر WAF.

ب. التلاعب بالطريقة (HTTP Method Tampering):
إذا كان المسار الإداري يحظر طلبات `POST /admin/user/delete`، يتم تجربة `GET` أو `PUT` أو `PATCH`.

ج. تعديل استجابة JSON & Mass Assignment:
تعديل حقل الصلاحية في الاستجابة (مثل `"isAdmin": true`) أو حقن حقل `"role": "admin"` ضمن طلبات التحديث.

د. إساءة استخدام الـ JWT (JWT Misconfiguration):
تعديل الـ Payload من `"role": "user"` إلى `"role": "admin"` واختبار خوارزمية `"alg": "none"`.

6. آليات الفحص التلقائي (Automated Testing with Autorize)

Burp Extension — Autorize: الأداة المعيارية لفحص التحكم بالوصول.
تعتمد على تصفح التطبيق بحساب مدير النظام (Administrator) بينما تكرر الأداة نفس الطلبات تلقائياً باستخدام Cookie حساب عادي (Low-Privilege Cookie) وطلب بدون كوكيز (Unauthenticated)، ثم مقارنة الاستجابات وطول المحتوى كاشفة عن الثغرة فوراً.

7. قواعد البرمجة الآمنة والمعالجة (Secure Remediation Guidelines)

تطبيق مبدأ القيد الصارم على مستوى الخادم لكل طلب (Server-Side Enforcement):

❌ نموذج كود غير آمن (غياب التحقق من الدور في الخادم):

```
@PostMapping("/admin/delete-user") public ResponseEntity deleteUser(@RequestParam String userId) { // الخادم ينفذ الحذف مباشرة بمجرد معرفة المسار دون فحص صلاحية المنفذ! userService.deleteById(userId); return ResponseEntity.ok().build(); }
```

✅ نموذج كود آمن (التحقق المركزي من الصلاحيات و RBAC):

```
@PostMapping("/admin/delete-user") @PreAuthorize("hasRole('ROLE_ADMIN')") // استخدام فحص الصلاحية المركزي public ResponseEntity deleteUser(@RequestParam String userId, Authentication auth) { if (auth == null || !auth.getAuthorities().contains(new SimpleGrantedAuthority("ROLE_ADMIN"))) { throw new AccessDeniedException("403 Forbidden: Insufficient Permissions"); } userService.deleteById(userId); return ResponseEntity.ok().build(); }
```